Code Supplement: Implementation and User Guide for Auctions with Participant Audits

1 Overview

This supplement documents the code behind the numerical results of the paper: how the package is organized, how experiments are run, and what they output. The methods it implements (type distributions, exact computation of the four mechanisms, threshold-response ascent and grid search, checks) are described in the empirical supplement, Section 1. The code is the Python package `PA_auctions` in this repository. The configuration keys, outputs and Python interface are documented in `docs/user_guide.md`, and `PA_auctions/README.md` documents the methods at the level of functions.

2 Package Layout

`run.py` is the command-line entry point. `scripts/` holds the data scripts of the paper and the empirical supplement and the previews of the synthetic distributions in `docs/distributions/`. `tests/` holds the test suite (`pytest`).

3 Running Experiments

A run is described by a TOML file and started with `python run.py CONFIG.toml`. Every list in the file is a set of values, and the run computes every combination; combinations that cannot run, such as the relaxation start with $k \geq n$, are skipped and listed. There are two kinds of run.

- `optimize`: the sets are distributions, markets, objectives (VCG-PA or IVCG-PA welfare), and optimizers. Each optimizer has a method (`threshold_response`, `grid`, `genetic`, `genetic+threshold_response`, `relaxation`), its parameters, and its own list of threshold forms; a form the method cannot search is a configuration error. Given threshold functions (below) may be added as benchmarks; each is evaluated in every market.
- `evaluate`: the sets are distributions, markets, and given threshold functions. A threshold function is given as a polynomial, a constant, a tabulated function, a welfare cutoff (a number, or the ex ante welfare cutoff), the threshold function of the symmetric relaxation, a saved `tau.json`, or every optimum of an earlier run.

Markets are either all pairs of the lists $n \in N$, $k \in K$, or an explicit list of pairs (n, k) (key `pairs`). The option `continuous` (default true) selects evaluation on continuous versions (empirical supplement, Section 1.2). Before running, the code prints the plan and a time estimate, calibrated by timing exact evaluations on each distribution. A search stopped by the time limit (default 60 minutes per combination) returns the best τ found so far, and the check `search_stop` fails for that combination. Combinations run in parallel worker processes (default: number of cores minus 2).

Workers beyond one per combination run the parallel stages within a combination: search starts and Nelder–Mead starts. Table 2 lists the command-line options.

Reproducibility. The run record `run.toml` stores the following.
- The resolved configuration, with every default explicit.
- The random seeds.
- The git commit, and whether tracked files had uncommitted changes. If they had, their diff is saved as `code.patch`.
- Package versions and the host (local machine, or a batch job with its job id and slots).
- Start and end times, and the runtime of each stage.

Rerunning the saved `config.toml` at the recorded commit reproduces the run. Figures are drawn only from saved files, so `python run.py --report RUN` redraws them without recomputation.

Tests. `python -m pytest` runs the test suite. `configs/smoke_test.toml` runs the smallest experiment through every stage on coarse grids, in about 15 seconds; its results are not meaningful.

Experiments of the paper. The paper and the empirical supplement use one run, of the configuration `configs/paper_run.toml`: four distributions, 20 markets, 716 combinations, about 51 minutes on 28 processor cores. On a cluster, it runs unchanged as a batch job, with `--yes` and `--workers N` for the number of cores `N` of the job. With its run folder `RUN`,

```
python scripts/web27_paper_data.py RUN OUT
python scripts/empirical_supplement_data.py RUN empirical_supplement
```

write the tables, curves, type backgrounds and axis limits from which the paper and the empirical supplement draw their figures and tables in LaTeX. The first writes the data of the figures of the paper's Section 7 and Appendix A to `OUT`; the second writes `empirical_supplement/data/`, from which `empirical_supplement_v2.tex` compiles.

4 Outputs

A run writes one folder, `runs/<date>_<time>_<kind>_<name>`, with the files of table 3. Each combination has a sub-folder

`<distribution>/z<ζ>/n<n>-k<k>/<design>`,

where ζ is an externality scale (1 in the paper). The design label is `<objective>_<form>_<optimizer>` for an optimized τ and `tau-<label>` for a given one. A comparison figure goes in the `compare/` folder of the deepest level above every input it varies.

Previews and README files. Every figure has a WebP preview in a `thumbs/` folder next to it. The README file of each combination and of each `compare/` folder shows the previews, each linked to its PDF and preceded by a description.

Figures for a paper. The option `--paper` exports each panel of the selected figures as its own PDF at printed size, with fixed margins, so panels placed side by side align; legends and colorbars go to their own PDFs, and each figure gets a caption draft and a LaTeX `figure` environment. The figures and tables of the paper and the empirical supplement do not use these exports: they are drawn in LaTeX from the data written by the two scripts above.

Module	Implements
`type_distributions.py`	type distributions as lists of cells (piecewise uniform, atomic), the synthetic distributions, continuous and refined versions
`threshold_functions.py`	threshold functions $\tau(e)$ (picklable classes: polynomial, piecewise linear, cutoff, tabulated)
`threshold_forms.py`	parametric forms of τ searched by the optimizers (`nodes`, `poly<d>` with the clipped affine `poly1`, `cutoff`, `knots<K>`, `bins<K>`) and the ex ante welfare cutoff
`scoring_rules.py`	score-based auctions: VCG, VCG-PA, IVCG-PA, first best
`exact_computation.py`	exact expected outcomes in quantile space: bid segments, ironing, interim allocations, Gauss–Legendre integration; continuous bid segments
`marginal_contribution.py`	marginal contributions M^τ, rival welfare loss L^τ, Pigouvian charges
`threshold_response_ascent.py`	threshold-response ascent: starts, threshold responses, line search, L-BFGS-B refinement, per-iteration diagnostics
`grid_search.py`	exhaustive grid search over the best cutoff and the best clipped affine τ, with Nelder–Mead refinement
`genetic_search.py`	genetic search with Nelder–Mead refinement (not used in the paper)
`symmetric_relaxation.py`	linear program over symmetric participation rules and the threshold function read from it (the relaxation start of the ascent)
`threshold_evaluation.py`	entry point: outcomes of the four mechanisms for a distribution and τ, exact and simulated
`auction_simulation.py`	simulated auctions on sampled type profiles (common random numbers)
`empirical.py`, `smoothing.py`	populations given as a table of scores: split, truncation, kernel smoothing, bootstrap (not used in the paper)
`pipeline.py`, `parallel.py`	building blocks of one combination; process-level parallelism within a combination
`figures.py`	the figures of a run, drawn from saved files
`experiments/`	`config.py` (read, complete, validate and expand a TOML file), `runner.py` (plan, time estimate, parallel combinations), `compute.py` (one combination and its files), `report.py` (figures and README files), `provenance.py` (git, versions, host, seeds)

Table 1: Modules of `PA_auctions`.

Command	Does
`run.py CONFIG.toml`	plan, time estimate, confirmation, computation, figures, README files
`  --dry-run`	plan and time estimate only
`  --yes`	no confirmation (batch jobs)
`  --workers N`	number of worker processes
`  --no-report`	computation only, no figures
`run.py --report RUN`	figures and README files of an existing run, from its saved files
`  --paper [NAMES]`	figures for a paper: one PDF per panel (section 4)
`run.py --options`	distributions, forms, objectives, optimizers, threshold sources and all defaults

Table 2: Command-line options of `run.py`.

File	Contents
`README.md`	index: inputs, counts of check verdicts, results table with links to every figure, figure reference
`config.toml`, `run.toml`	configuration as given; resolved configuration and provenance
`code.patch`, `log.txt`	uncommitted changes of tracked code (if any); progress messages
`summary.csv`	one row per combination: inputs, exact outcomes of the four mechanisms, gains, shares of the first-best gain, revenue changes, runtime
`checks.csv`	one row per check and combination
`gap-decomposition.csv`	per distribution, market and class of threshold functions: the gaps $\mathrm{FB} - \mathrm{OPT}$, $\mathrm{OPT} - \mathrm{OPT}^{\mathrm{VCG}}$ and $\mathrm{OPT}^{\mathrm{VCG}} - \mathrm{OPT}^{\mathrm{VCG}}_{\mathcal{T}}$, each with the best values found
`tau.json`, `tau.csv`	τ (rebuildable, reusable in evaluate runs); $\tau(e)$ and both penalty functions at the externality nodes
`results.json`	every statistic of a combination, including search statistics and convergence traces
`first-order.csv`	per node and mechanism: threshold, X^{τ}, penalty, Pigouvian charge, marginal contribution, threshold-response gap, first-order violation
`data.npz`	arrays behind the figures: simulated outcomes, ironing segments, interim allocations, maps of M^{τ}, tried thresholds
`types.npz`	per distribution: the type density drawn behind the figures
`README.md` (combination)	results table, files, each figure with its description and preview, figures not drawn and why
`runs/index.csv`	one line per run: folder, kind, start, runtime, combinations, failures, failed checks, commit, host

Table 3: Files of a run and of each combination.

Figure	Shows	Drawn
1 types, τ, penalties	type distribution, τ, abstention region, excluded externalities, zero-welfare line $v = -e$, bids pooled by IVCG-PA; penalty functions	always
2 expected outcomes	expected welfare as bidder net utility + revenue + penalties + externality part, with the share of the first-best gain; changes over VCG of each quantity	always
3 outcome distributions	per quantity: per-auction distribution of each mechanism in simulated auctions, and of the paired difference from VCG (with the share of exact zeros); exact means marked	always
4 gain scatter	per simulated auction: captured gain against achievable gain; revenue change against welfare change	always
5 ironing	$\hat{w}^\tau$ and $\bar{\omega}^\tau$ in quantile space, Ψ and its convex hull, scores against bids, interim allocations	ironing binds
6 marginal contribution	maps of $M^\tau(e, v)$ with the zero contour and τ; penalty against Pigouvian charge; M^τ at threshold types	always
search: tried	tried thresholds behind the best one	optimize
search: convergence	training objective against time, per start and stage	optimize
search: iterations	per iteration: share of the first-best gain, distance to the best welfare found, stationarity, first-order violation, threshold-response gap	optimize
compare: τ	τ of combinations that differ in one input (market, form, optimizer, objective, threshold); τ by n at fixed k, by k at fixed n, and by k at fixed n/k	≥ 2 such
compare: share	share of the first-best gain of every combination	always

Table 4: Figures of a run. All are PDF. Each mechanism has a fixed color and line style, chosen to remain distinguishable under color-vision deficiencies.